



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 02

NOMBRE COMPLETO: Tavera Castillo David Emmanuel

N° de Cuenta: 320054831

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 04/09/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

En el primer ejercicio dibujamos as iniciales de nuestro nombre (en mi caso TCD) asignándole a cada letra un color diferente.

Haciendo uso de las coordenadas ya utilizadas en la practica anterior las agregamos a una función donde podemos asignar a cada triangulo un color así generando las letras de distintos colores.

```
void CrearLetrasyFiguras()
{
    GLfloat vertices_letras[] = {

        //T
        //X      Y      Z      R      G      B
        -0.9f,  0.8f,  0.0f,  1.0f,  1.0f,  1.0f,
        -0.3f,  0.8f,  0.0f,  1.0f,  1.0f,  1.0f,
        -0.3f,  0.6f,  0.0f,  1.0f,  1.0f,  1.0f,

        -0.9f,  0.8f,  0.0f,  1.0f,  1.0f,  1.0f,
        -0.9f,  0.6f,  0.0f,  1.0f,  1.0f,  1.0f,
        -0.3f,  0.6f,  0.0f,  1.0f,  1.0f,  1.0f,

        -0.7f,  0.6f,  0.0f,  1.0f,  1.0f,  1.0f,
        -0.5f,  0.6f,  0.0f,  1.0f,  1.0f,  1.0f,
        -0.7f,  0.1f,  0.0f,  1.0f,  1.0f,  1.0f,

        -0.5f,  0.6f,  0.0f,  1.0f,  1.0f,  1.0f,
        -0.5f,  0.1f,  0.0f,  1.0f,  1.0f,  1.0f,
        -0.7f,  0.1f,  0.0f,  1.0f,  1.0f,  1.0f,

        MeshColor *letras = new MeshColor();
        letras->CreateMeshColor(vertices_letras,396);
        meshColorList.push_back(letras);
    }
```

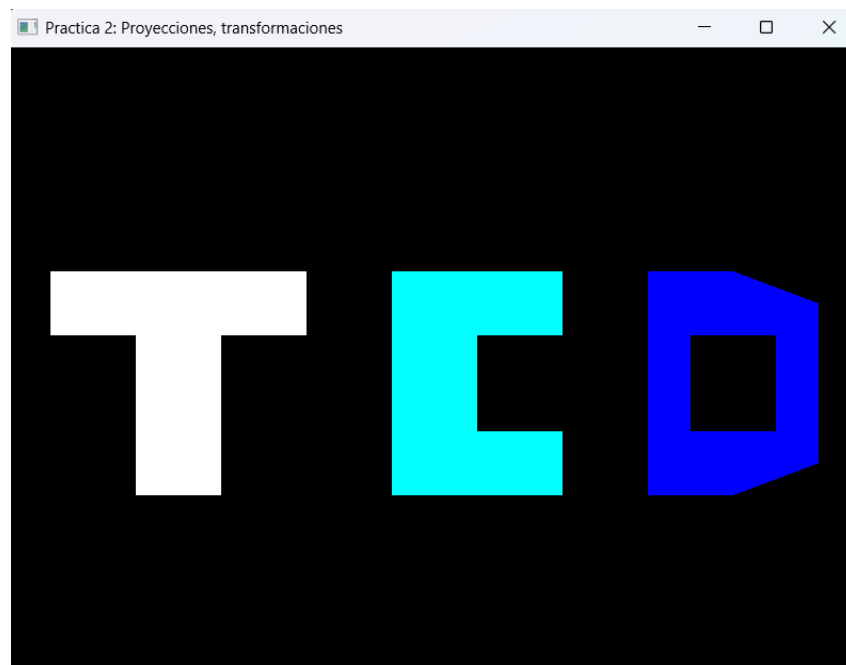
Para imprimirlo en la ventana necesitamos de las siguientes líneas de código.

```
shaderList[1].useShader();
uniformModel = shaderList[1].getModelLocation();
uniformProjection = shaderList[1].getProjectLocation();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -0.5f, -2.0f));

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshColorList[0]->RenderMeshColor();
```

Dádonos como resultado lo siguiente:



El segundo ejercicio consta de crear una casa con pirámides y cubos, para esto vamos a crear shaders de la siguiente forma:

```
#version 330
layout (location =0) in vec3 pos;
layout (location =1) in vec3 color;
out vec4 vColor;
uniform mat4 model;
uniform mat4 projection;
void main()
{
    gl_Position=projection*model*vec4(pos.x,pos.y,pos.z,1.0f);
    vColor=vec4(0.0f, 1.0f, 0.0f,1.0f);
}
```

Lo único que cambia en todos los shaders es el color que devuelve.

Necesitaremos importarlo en el archivo main con la siguiente sentencia:

```
static const char* vShaderColorRed = "shaders/shadercolorRed.vert";
```

Con las siguientes líneas de código lo agregamos a un arreglo para su utilización.

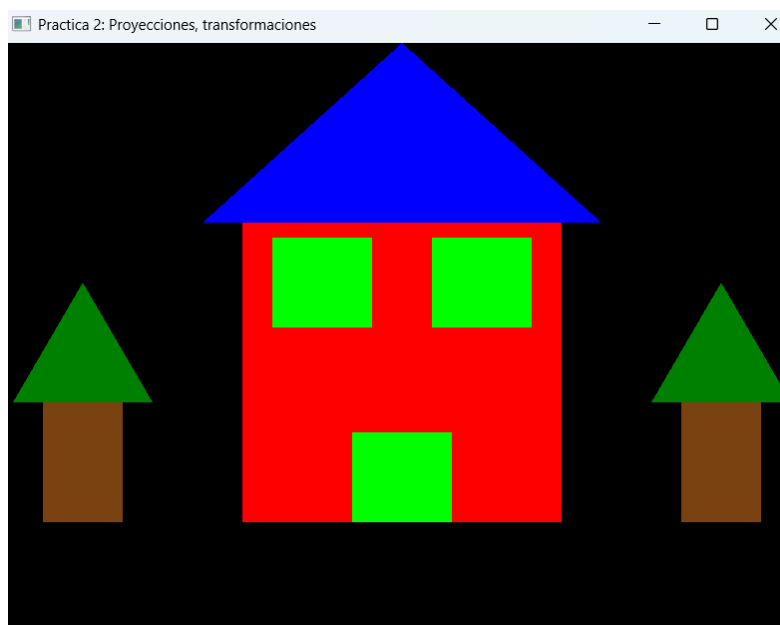
```
Shader* shaderRed = new Shader();
shaderRed->CreateFromFiles(vShaderColorRed, fShader);
shaderList.push_back(*shaderRed);
```

Para poder imprimir las figuras necesitamos primer instanciar el color y después la figura de la siguiente manera:

```
shaderList[3].useShader();
uniformModel = shaderList[3].getModelLocation();
uniformProjection = shaderList[3].getProjectLocation();

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.7f, -4.0f));
model = glm::scale(model, glm::vec3(1.0f, 0.60f, 0.30f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshList[0]-->RenderMesh();
```

Finalizando obtenemos la siguiente figura:



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

Durante la realización hubo un problema de compatibilidad con los shaders ya que aun con la extensión .vert, VisualStudio lo detectaba como un .cpp, para la solución modifique la configuración para evitar este error de lectura.

3.- Conclusión:

Se completo la practica de manera correcta aun con dificultades de configuración, se comprendió la diferencia de uso de shaders además de como se visualizan las proyecciones ortogonales y de perspectiva.

4.- Bibliografía en formato APA

LearnOpenGL - Shaders. (s. f.). [https://learnopengl.com/Getting started/Shaders](https://learnopengl.com/Getting%20started/Shaders)